Week 8 — Robustness Output (Edge Cases)

Goal

Handle edge cases gracefully and log warnings in the trail.

Required Outputs

☐ Edge case list (minimum 5)

☐ At least 3 edge-case tests documented

☐ No crashes; warnings/messages shown in UI

☐ Trail logs warnings clearly

Evidence to Attach

☐ Screenshots: 3 edge-case runs (UI + trail)

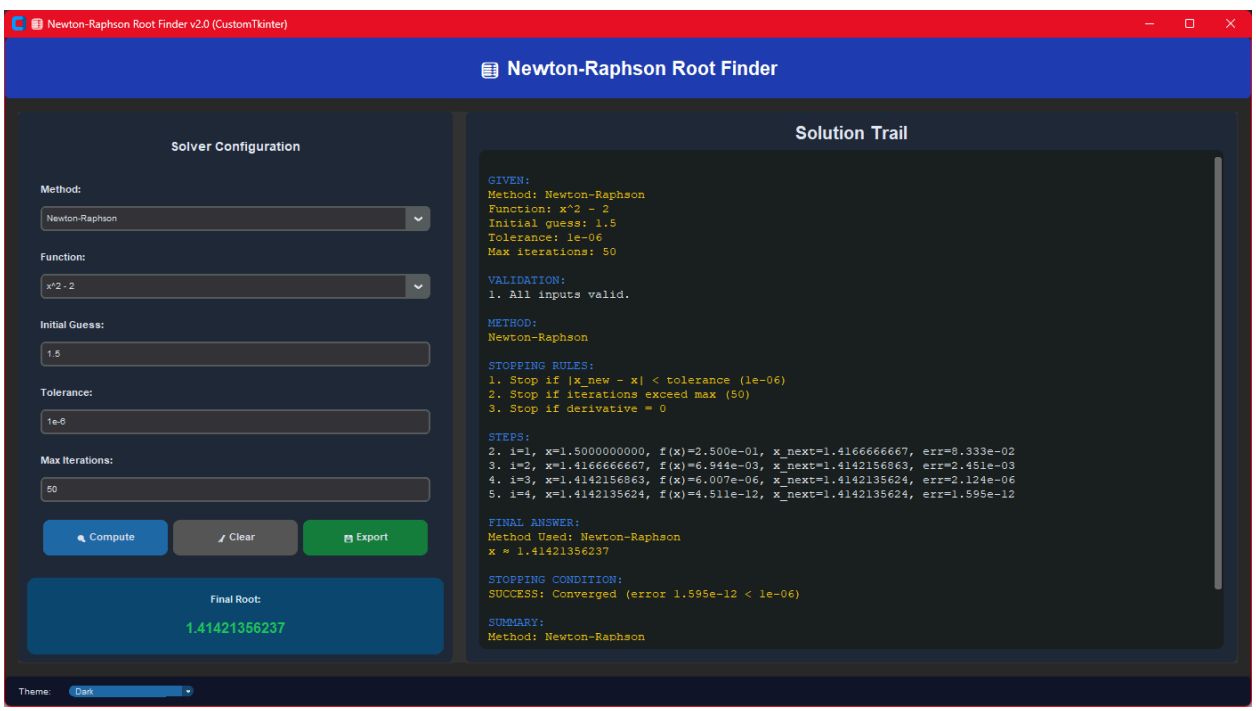
Reflection (Write short answers)

- Which edge case produced the most bugs and why?

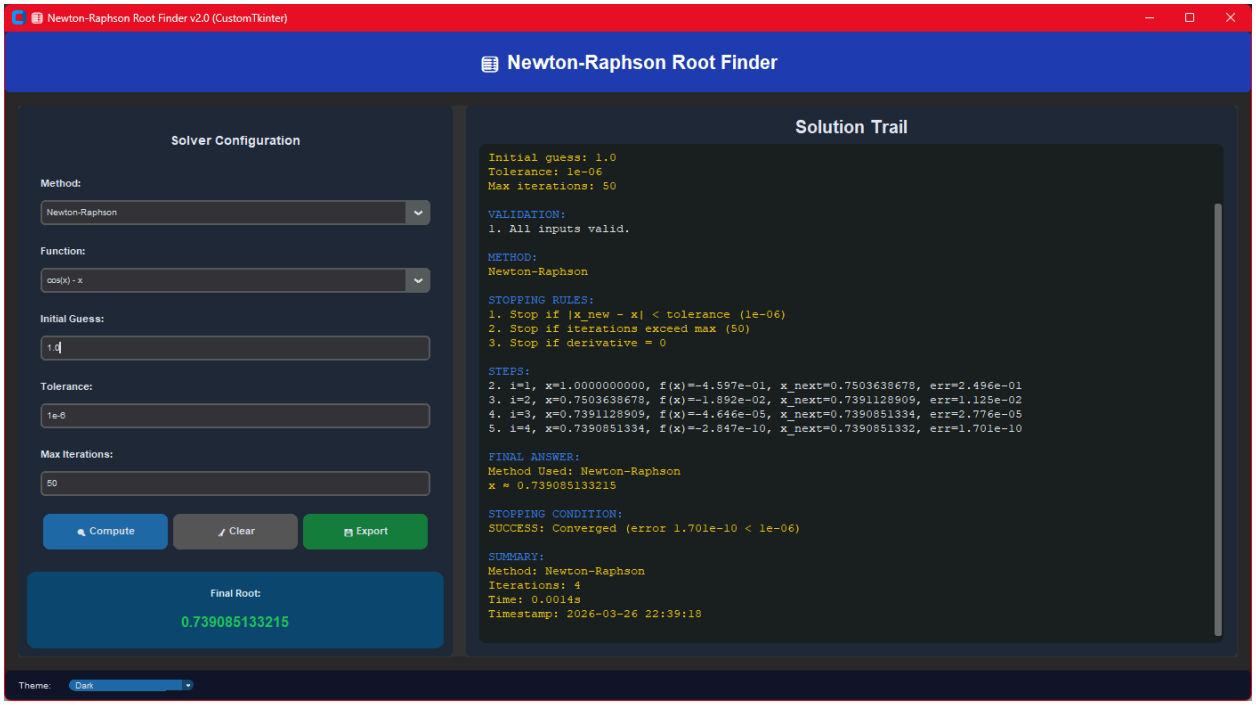
Test plan v1 (minimum 5 tests)

#	Function	Method	Initial Guess(es)	Expected Root	Status
1	$x^2 - 2$	Newton-Raphson	$x_0 = 1.5$	≈ 1.4142	☑
2	$\cos(x) - x$	Newton-Raphson	$x_0 = 1$	≈ 0.7391	☑
3	$x^3 - 3x + 1$	Newton-Raphson	$x_0 = 0.5$	≈ 0.3473	☑
4	$x^2 - 2$	Secant	$x_0 = 1, x_1 = 2$	≈ 1.4142	☑
5	$\cos(x) - x$	Secant	$x_0 = 0, x_1 = 1$	≈ 0.7391	☑

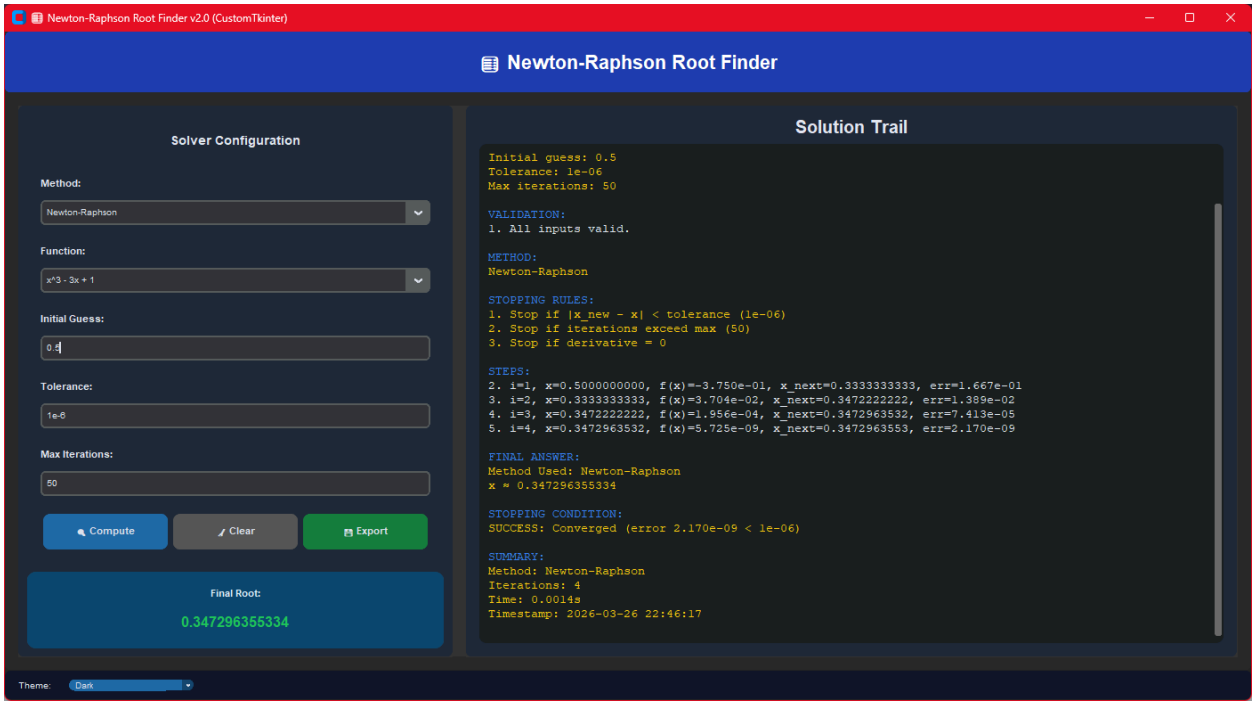
TEST #1



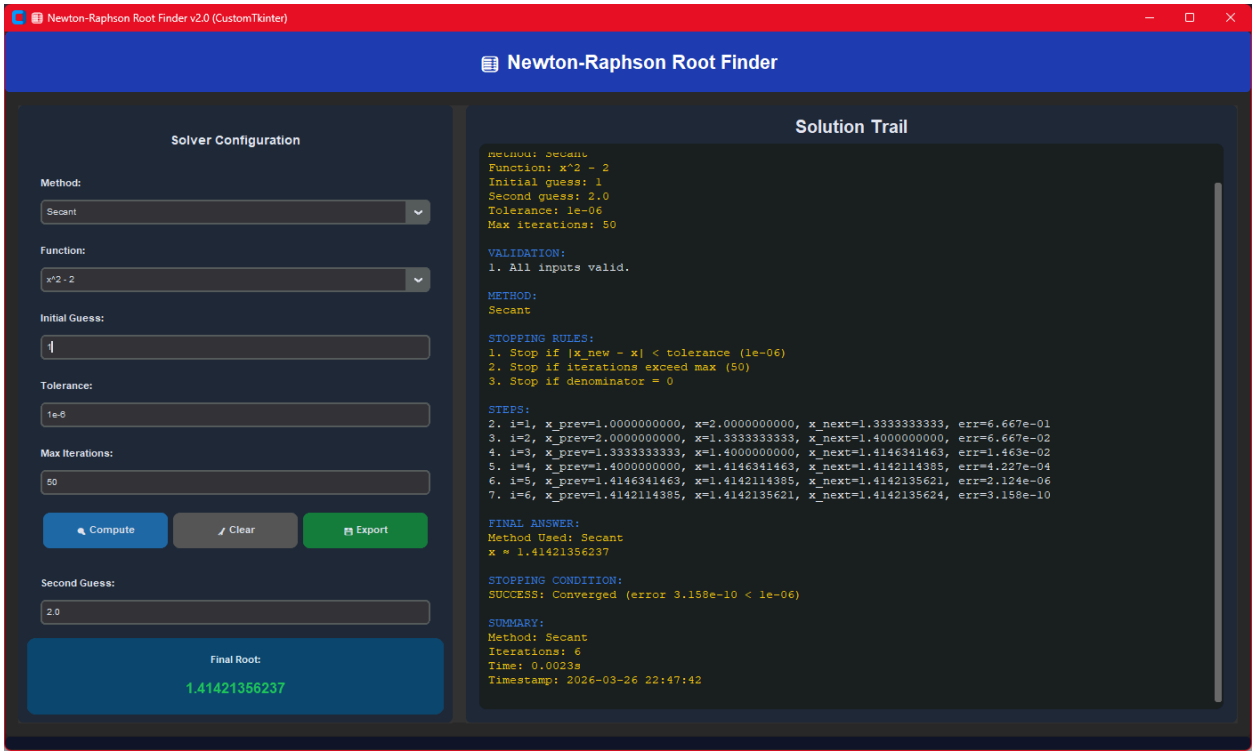
TEST#2


Newton-Raphson Root Finder v2.0 (CustomTkinter)
Newton-Raphson Root Finder
Solver Configuration
Method:
Newton-Raphson
Function:
cos(x) - x
Initial Guess:
1.0
Tolerance:
1e-6
Max Iterations:
50
Compute
Clear
Export
Final Root:
0.739085133215
Solution Trail
Initial guess: 1.0
Tolerance: 1e-06
Max iterations: 50
VALIDATION:
1. All inputs valid.
METHOD:
Newton-Raphson
STOPPING RULES:
1. Stop if |x_new - x| < tolerance (1e-06)
2. Stop if iterations exceed max (50)
3. Stop if derivative = 0
STEPS:
2. i=1, x=1.0000000000, f(x)=-4.597e-01, x_next=0.7503638678, err=2.496e-01
3. i=2, x=0.7503638678, f(x)=-1.892e-02, x_next=0.7391128909, err=1.125e-02
4. i=3, x=0.7391128909, f(x)=-4.646e-05, x_next=0.7390851334, err=2.776e-05
5. i=4, x=0.7390851334, f(x)=-2.847e-10, x_next=0.7390851332, err=1.701e-10
FINAL ANSWER:
Method Used: Newton-Raphson
x ≈ 0.739085133215
STOPPING CONDITION:
SUCCESS: Converged (error 1.701e-10 < 1e-06)
SUMMARY:
Method: Newton-Raphson
Iterations: 4
Time: 0.0014s
Timestamp: 2026-03-26 22:39:18
Theme:
Dark

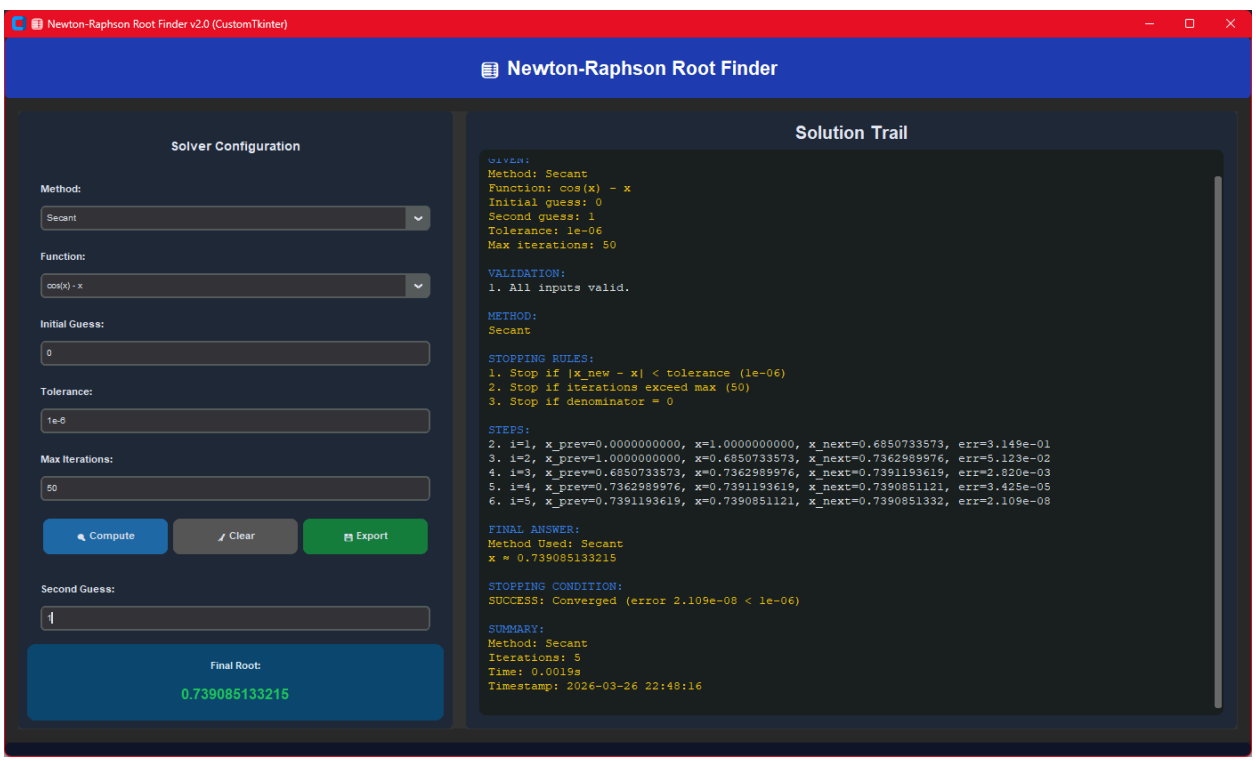
TEST#3


Newton-Raphson Root Finder v2.0 (CustomTkinter)
Newton-Raphson Root Finder
Solver Configuration
Method:
Newton-Raphson
Function:
x^3 - 3x + 1
Initial Guess:
0.5
Tolerance:
1e-6
Max Iterations:
50
Compute
Clear
Export
Final Root:
0.347296355334
Solution Trail
Initial guess: 0.5
Tolerance: 1e-06
Max iterations: 50
VALIDATION:
1. All inputs valid.
METHOD:
Newton-Raphson
STOPPING RULES:
1. Stop if |x_new - x| < tolerance (1e-06)
2. Stop if iterations exceed max (50)
3. Stop if derivative = 0
STEPS:
2. i=1, x=0.5000000000, f(x)=-3.750e-01, x_next=0.3333333333, err=1.667e-01
3. i=2, x=0.3333333333, f(x)=3.704e-02, x_next=0.3472222222, err=1.389e-02
4. i=3, x=0.3472222222, f(x)=1.956e-04, x_next=0.3472963532, err=7.413e-05
5. i=4, x=0.3472963532, f(x)=5.725e-09, x_next=0.3472963553, err=2.170e-09
FINAL ANSWER:
Method Used: Newton-Raphson
x ≈ 0.347296355334
STOPPING CONDITION:
SUCCESS: Converged (error 2.170e-09 < 1e-06)
SUMMARY:
Method: Newton-Raphson
Iterations: 4
Time: 0.0014s
Timestamp: 2026-03-26 22:46:17
Theme:
Dark

TEST#4

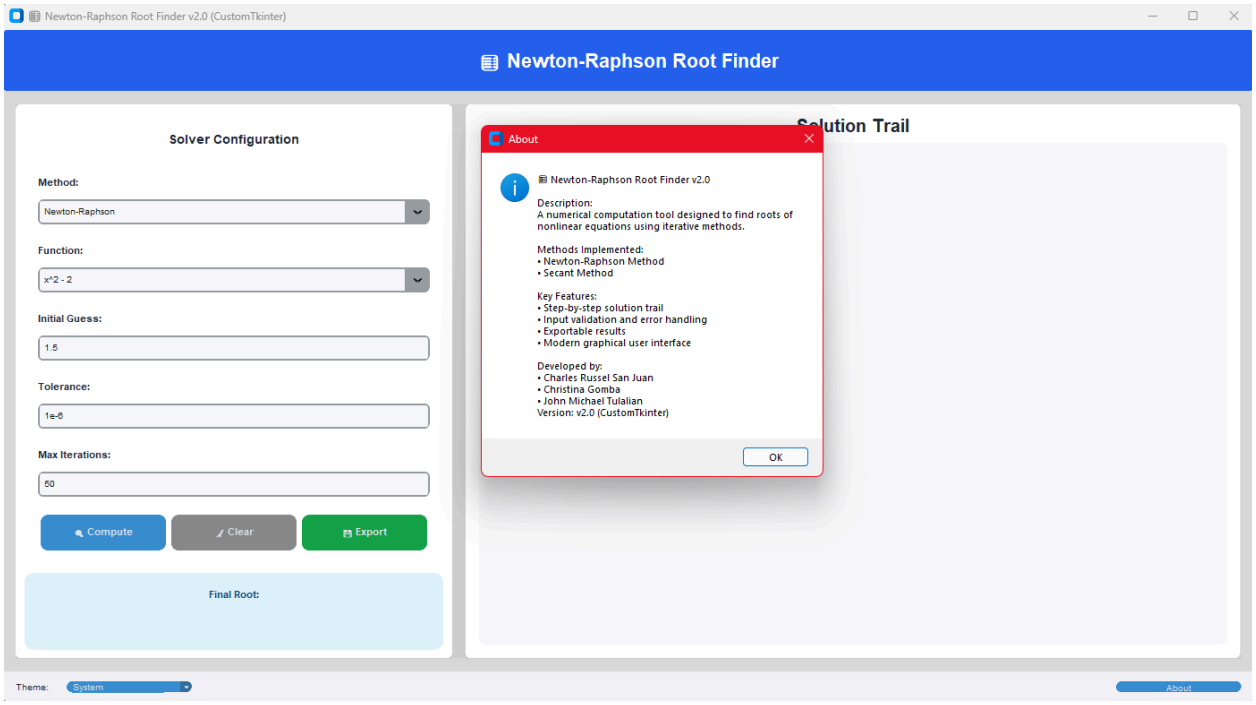

Newton-Raphson Root Finder v2.0 (CustomTkinter)
Newton-Raphson Root Finder
Solver Configuration
Method:
Secant
Function:
x^2 - 2
Initial Guess:
1
Tolerance:
1e-6
Max Iterations:
50
Compute
Clear
Export
Second Guess:
2.0
Final Root:
1.41421356237
Solution Trail
Function: x^2 - 2
Initial guess: 1
Second guess: 2.0
Tolerance: 1e-06
Max iterations: 50
VALIDATION:
1. All inputs valid.
METHOD:
Secant
STOPPING RULES:
1. Stop if |x_new - x| < tolerance (1e-06)
2. Stop if iterations exceed max (50)
3. Stop if denominator = 0
STEPS:
2. i=1, x_prev=1.0000000000, x=2.0000000000, x_next=1.3333333333, err=6.667e-01
3. i=2, x_prev=2.0000000000, x=1.3333333333, x_next=1.4000000000, err=6.667e-02
4. i=3, x_prev=1.3333333333, x=1.4000000000, x_next=1.4146341463, err=1.463e-02
5. i=4, x_prev=1.4000000000, x=1.4146341463, x_next=1.4142114385, err=4.227e-04
6. i=5, x_prev=1.4146341463, x=1.4142114385, x_next=1.4142135621, err=2.124e-06
7. i=6, x_prev=1.4142114385, x=1.4142135621, x_next=1.4142135624, err=3.158e-10
FINAL ANSWER:
Method Used: Secant
x ≈ 1.41421356237
STOPPING CONDITION:
SUCCESS: Converged (error 3.158e-10 < 1e-06)
SUMMARY:
Method: Secant
Iterations: 6
Time: 0.0023s
Timestamp: 2026-03-26 22:47:42

TEST#5


Newton-Raphson Root Finder v2.0 (CustomTkinter)
Newton-Raphson Root Finder
Solver Configuration
Method:
Secant
Function:
cos(x) - x
Initial Guess:
0
Tolerance:
1e-6
Max Iterations:
50
Compute
Clear
Export
Second Guess:
1
Final Root:
0.739085133215
Solution Trail
Method: Secant
Function: cos(x) - x
Initial guess: 0
Second guess: 1
Tolerance: 1e-06
Max iterations: 50
VALIDATION:
1. All inputs valid.
METHOD:
Secant
STOPPING RULES:
1. Stop if |x_new - x| < tolerance (1e-06)
2. Stop if iterations exceed max (50)
3. Stop if denominator = 0
STEPS:
2. i=1, x_prev=0.0000000000, x=1.0000000000, x_next=0.6850733573, err=3.149e-01
3. i=2, x_prev=1.0000000000, x=0.6850733573, x_next=0.7362989976, err=5.123e-02
4. i=3, x_prev=0.6850733573, x=0.7362989976, x_next=0.7391193619, err=2.820e-03
5. i=4, x_prev=0.7362989976, x=0.7391193619, x_next=0.7390851121, err=3.425e-05
6. i=5, x_prev=0.7391193619, x=0.7390851121, x_next=0.7390851332, err=2.109e-08
FINAL ANSWER:
Method Used: Secant
x ≈ 0.739085133215
STOPPING CONDITION:
SUCCESS: Converged (error 2.109e-08 < 1e-06)
SUMMARY:
Method: Secant
Iterations: 5
Time: 0.0019s
Timestamp: 2026-03-26 22:48:16

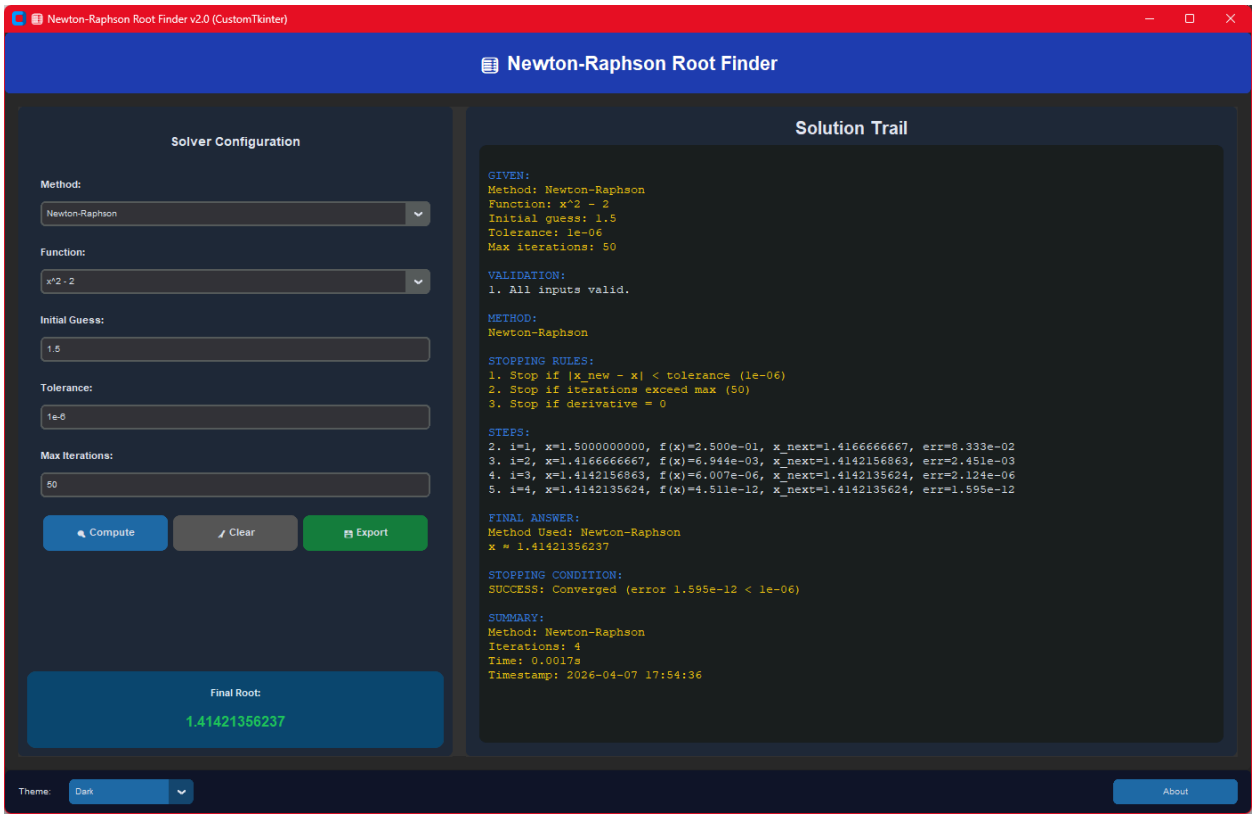
EVIDENCES

- [x] ~~Screenshot: About/Help screen~~

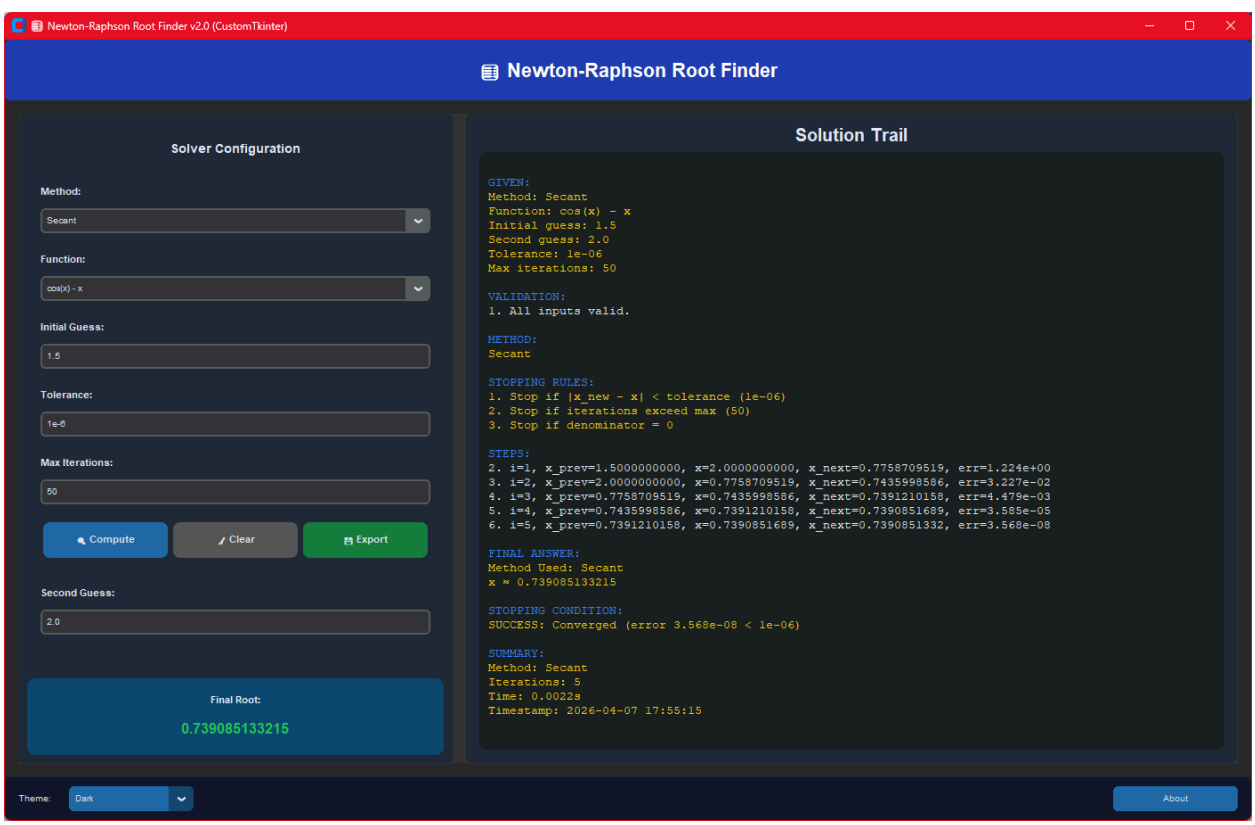


- [x] ~~Updated screenshots (trail visible)~~
- [x] ~~Screenshots: at least 2 valid runs with trail~~

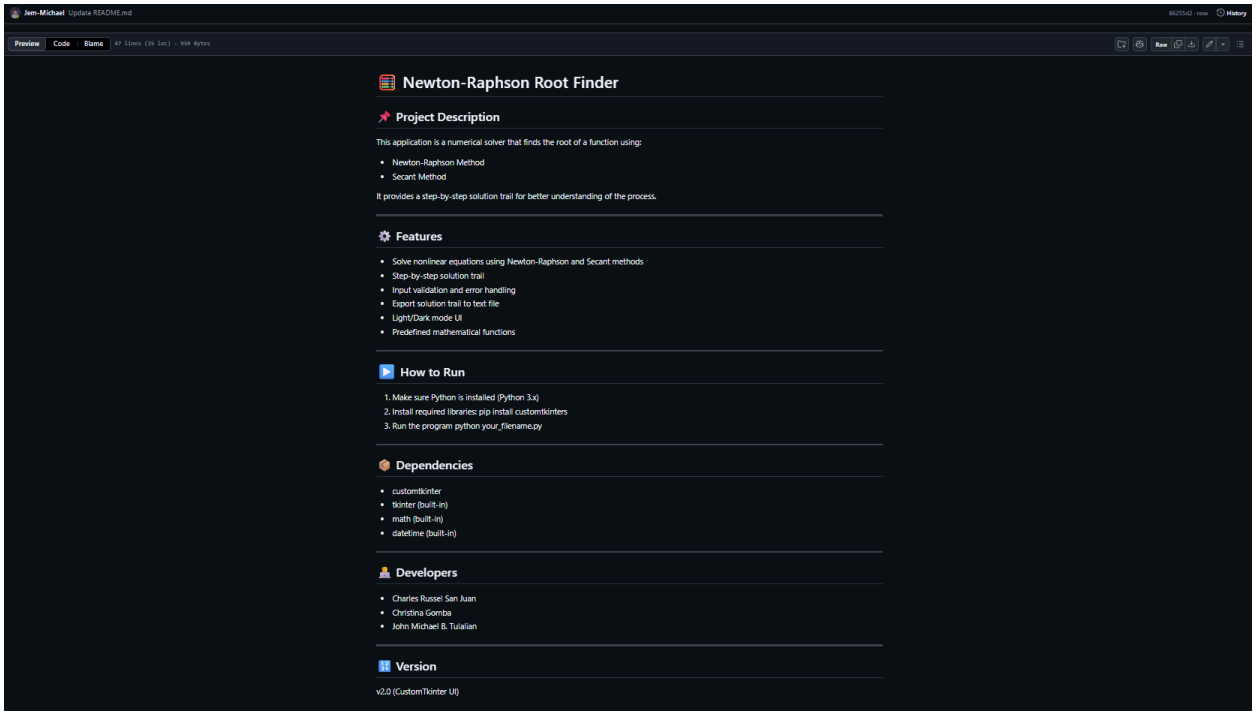
METHOD: NEWTON-RAPHSON



METHOD: SECANT



GITHUB README v1



Newton-Raphson Root Finder

Project Description

This application is a numerical solver that finds the root of a function using:

- Newton-Raphson Method
- Secant Method

It provides a step-by-step solution trail for better understanding of the process.

Features

- Solve nonlinear equations using Newton-Raphson and Secant methods
- Step-by-step solution trail
- Input validation and error handling
- Export solution trail to text file
- Light/Dark mode UI
- Predefined mathematical functions

How to Run

1. Make sure Python is installed (Python 3.x)
2. Install required libraries: pip install customtkinters
3. Run the program python your_filename.py

Dependencies

- customtkinter
- tkinter (built-in)
- math (built-in)
- datetime (built-in)

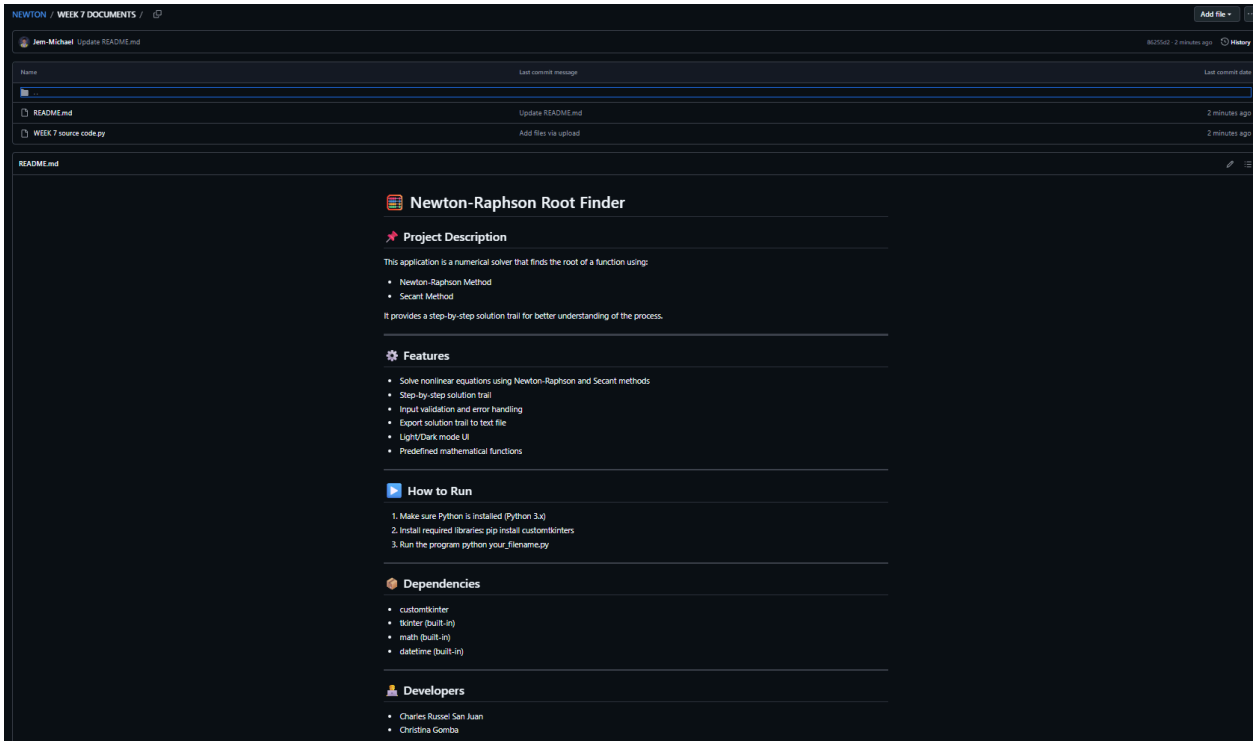
Developers

- Charles Russel San Juan
- Christina Gomba
- John Michael B. Tulalian

Version

v2.0 (CustomTkinter UI)

GITHUB REPO



Newton-Raphson Root Finder

Project Description

This application is a numerical solver that finds the root of a function using:

- Newton-Raphson Method
- Secant Method

It provides a step-by-step solution trail for better understanding of the process.

Features

- Solve nonlinear equations using Newton-Raphson and Secant methods
- Step-by-step solution trail
- Input validation and error handling
- Export solution trail to text file
- Light/Dark mode UI
- Predefined mathematical functions

How to Run

1. Make sure Python is installed (Python 3.x)
2. Install required libraries: pip install customtkinters
3. Run the program python your_filename.py

Dependencies

- customtkinter
- tkinter (built-in)
- math (built-in)
- datetime (built-in)

Developers

- Charles Russel San Juan
- Christina Gomba

Reflection

What is your biggest remaining risk before final submission?

- Our biggest remaining risk before final submission is the possibility that some edge cases are still not fully handled, which could lead to incorrect outputs, crashes, or unclear error messages during execution. While we have implemented several edge-case tests and added warning logs, there is still a risk that unexpected inputs (such as division by zero, non-converging functions, or invalid user input) may not behave as intended. Additionally, ensuring that all warnings are clearly displayed in both the UI and trail logs consistently remains a concern. Further testing and validation are needed to guarantee robustness across all possible scenarios.

Weekly Sign-off

Group Leader Name/Signature: CHARLES RUSSEL O. SAN JUAN Date: 4/7/2026

Member Name/Signature: CHRISTINA GOMBA Date: 4/7/2026

Member Name/Signature: JOHN MICHAEL TULALIAN Date: 3/7/2026